

PROYECTO AGROVOLTAICO · TEC COSTA RICA

AgroVoltaic

Arquitectura del sistema

Del sensor en el campo al agente que responde preguntas

Plataforma de datos del sistema agrovoltaiico de San Carlos: el ETL que estandariza diecinueve meses de CSV crudos, dos agentes con modelo de lenguaje (uno de análisis histórico y otro de pronóstico) y la consola con la que se depuran. Este documento describe cómo encajan las piezas, qué decisión hay detrás de cada una y qué queda pendiente.

25 de agosto de 2026

Índice

1. Panorama	2
1.1. El problema de origen	2
1.2. Dos regiones, sin base central	2
2. ETL de CSV a Supabase	4
2.1. Una sola lista, y todo lo demás se deduce de ella	4
2.2. Módulos	5
2.3. Idempotencia	5
2.4. Uso	5
3. Modelo de datos	6
3.1. Relaciones	6
3.2. Reglas de corrección	6
3.3. Calibración y desempeño	7
4. Agente Predictivo	7
4.1. La física	8
4.2. Estructura y dirección de dependencias	9
4.3. La ingesta ambiental	9
4.4. Endpoints	9
4.5. Frenos de consumo	9
4.6. Detección de anomalías	10
5. Agente Histórico	10
5.1. Una herramienta, un archivo, una pregunta	10
5.2. Endpoints	10
6. Consola de depuración	11
6.1. La regla de verificación	11
6.2. Camino de un request	11
6.3. Control de acceso	11
6.4. Dos formas de levantarla	12
7. Despliegue y operación	12
7.1. Por qué la plataforma vive en su propio servidor	12
7.2. Qué corre dónde	13
7.3. Qué se publica, y qué no	13
7.4. Por qué hay una réplica de AgroDash	13
7.5. Salud del sistema	14
7.6. Lección de observabilidad	14
7.7. Diagnóstico rápido	14
7.8. Pruebas	14
8. Seguridad	15
8.1. Secretos necesarios	15
8.2. Pendiente de seguridad	16
9. Estado actual	16
9.1. Volúmenes actuales	16
9.2. Decisiones que conviene no visitar	16

1 Panorama

AgroVoltaic es la plataforma de datos del sistema agrovoltaico del TEC en San Carlos (Costa Rica): paneles solares bifaciales instalados sobre cultivo. El repositorio es un *monorepo* con cuatro piezas que se pueden levantar por separado.

Pieza	Dónde vive	Qué resuelve
ETL de CSV	src/agrovoltaic	Estandarizar 19 meses de CSV crudos del inversor y los sensores, y cargarlos a Supabase de forma idempotente.
Agente de pronóstico	agente-pronostico/	Pronosticar irradiancia y humedad de suelo con anclaje físico; el LLM sólo orquesta.
Agente Histórico	agente-historico/	Responder preguntas sobre el histórico fotovoltaico ya limpio, componiendo herramientas atómicas.
Consola de depuración	mvp-debugger/	Ver la traza completa de cada agente y cruzar cada número contra los datos.

1.1 El problema de origen

Los datos no tenían estándar. El EDA sobre los 285 CSV encontró **13 esquemas distintos**: nombres de columna que cambian entre cortos (vpv1), largos (Voltaje PV1 [V]) y snake_case; erratas persistentes (Energì con acento grave en 72 archivos, P0Tencia, Corriente PV2[A] sin espacio); filas de sensores distintos intercaladas en el mismo archivo; irradiancia sin calibrar con un *offset* nocturno constante de $-38,845$; temperaturas saturadas en 85°C (el código de error clásico del DS18B20 desconectado); intervalos de muestreo que van de 2 segundos a 5 minutos según la época; y dos huecos largos de 126 y 71 días.

Regla rectora del tratamiento de datos

Validada con Leo Cardinale el 2026-08-10: **el dato crudo se guarda tal cual en la base; toda corrección vive en una capa de análisis** (vistas SQL) que genera variables nuevas. Ninguna transformación destructiva en la ingesta. Esta regla reemplazó las decisiones previas de convertir 85°C a NULL, forzar el *offset* a cero y remuestrear todo a 5 minutos durante la carga.

1.2 Dos regiones, sin base central

El proyecto abarca dos sitios y **no** se fusionan sus bases:

- **Cartago**: recolección automática hacia **AgroDash** (PostgreSQL, app Rust/Axum). Sensores de suelo y ambiente: humedad, conductividad, temperatura, irradiancia, PAR. No mide fotovoltaica.
- **San Carlos**: recolección semi-manual de CSV hacia **Supabase**. Es la región fotovoltaica.

El matiz que define la arquitectura: **San Carlos está partido entre las dos bases**. Lo eléctrico (voltaje, corriente, potencia, energía, irradiancia del inversor) vive en Supabase; lo ambiental y de suelo de San Carlos vive en AgroDash, en cajas con sufijo SC. Como Cartago no mide fotovoltaica, no existe comparación cruzada PV contra PV: lo único común entre regiones son las variables ambientales.

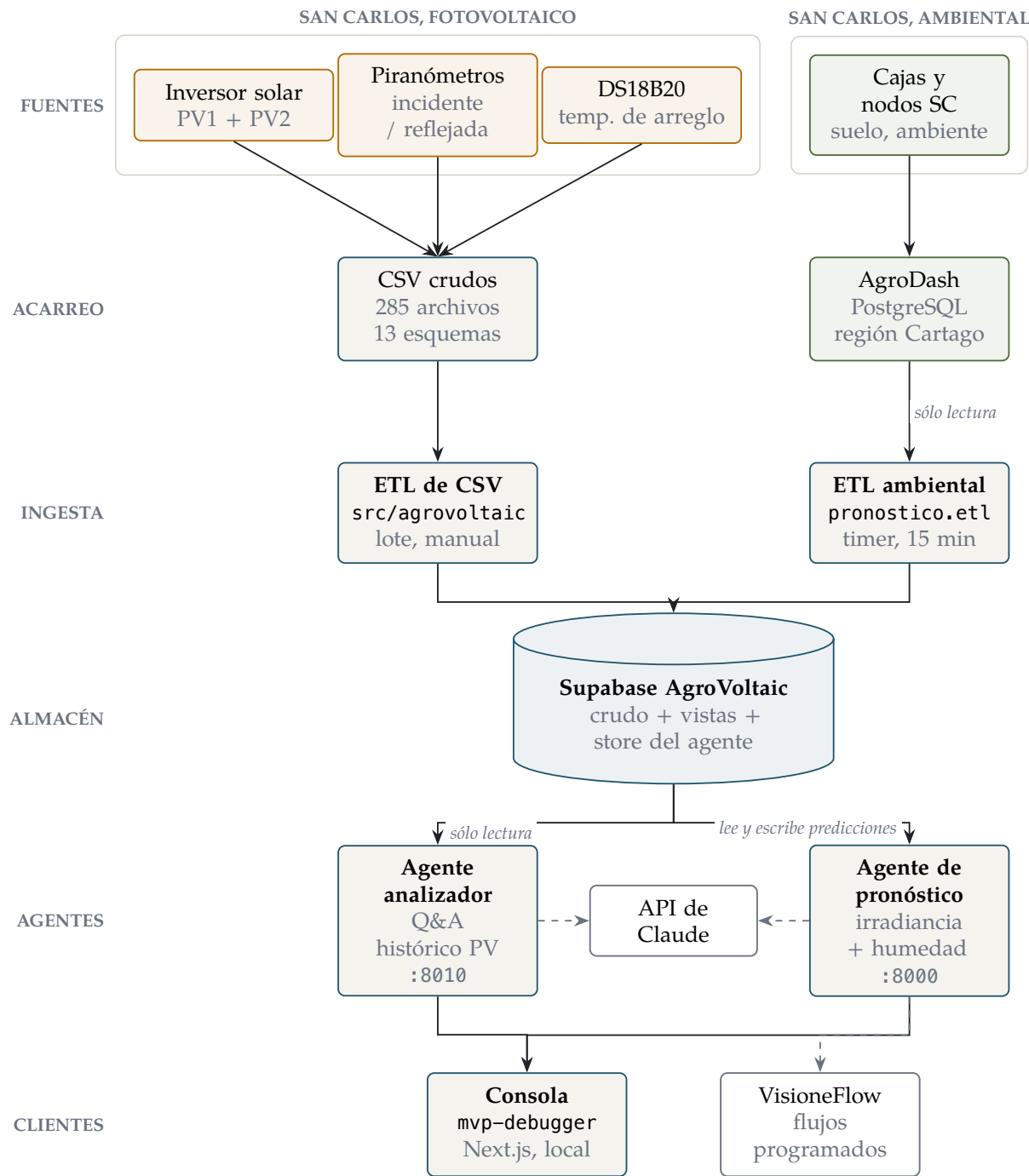


Figura 1: Arquitectura global. Dos rutas de ingesta independientes convergen en una sola Supabase, que es lo único que los agentes leen.

2 ETL de CSV a Supabase

Es el pipeline histórico: convierte la carpeta de CSV crudos en dos tablas normalizadas. No es un script de una sola vez, sino un proceso reproducible que se vuelve a correr cada vez que aparecen archivos nuevos.

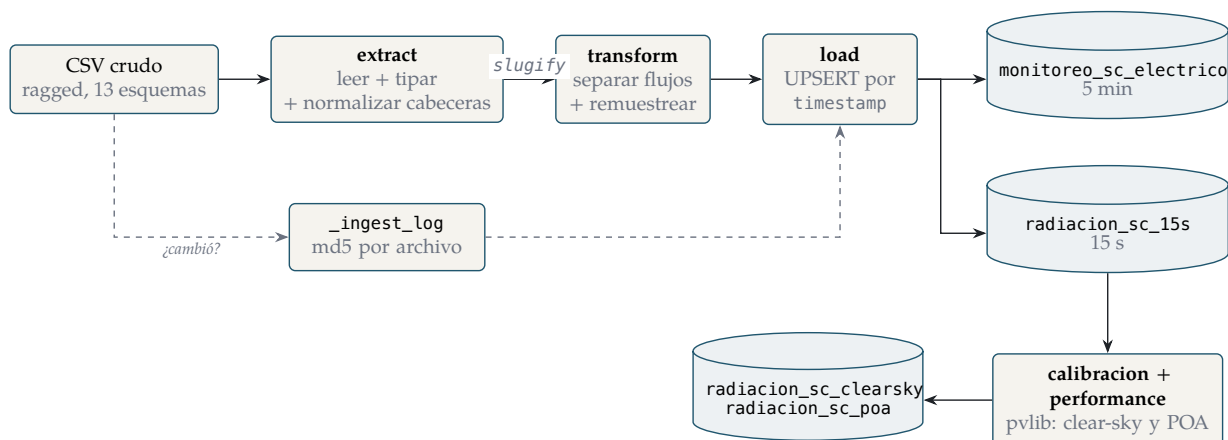


Figura 2: Pipeline de CSV. El `_ingest_log` decide qué archivos ni siquiera se abren; la capa solar (pvlb) se refresca al final de cada corrida.

2.1 Una sola lista, y todo lo demás se deduce de ella

El problema de fondo de estos CSV es que el mismo dato viene escrito de muchas maneras distintas. El voltaje del primer arreglo aparece como `vpv1` en unos archivos, como `Voltaje PV1 [V]` en otros y como `voltaje_pv1_v` en los más nuevos, y a eso hay que sumarle las erratas y los acentos que se fueron colando. En total, unas setenta formas distintas de nombrar apenas una docena de cosas.

La salida fácil sería escribir a mano la lista de nombres buenos y repetirla en cada lugar donde hace falta: al crear las tablas, al insertar, al agrupar por intervalos de tiempo. Es también la forma más segura de que dentro de seis meses alguien agregue una variable en tres de esos cuatro lugares, se olvide del cuarto, y nadie se entere hasta que los números no cuadren.

Por eso acá hay **un solo lugar** donde se declara qué se mide: un diccionario con una entrada por cada cosa medida, que dice cuál es su nombre bueno. Todo lo demás sale de ahí, incluido el SQL que crea la base de datos, que es un archivo **generado** y no se escribe a mano.

Para quien tenga que trabajar con esto mañana, la diferencia se nota en dos cosas:

- **Agregar una variable es escribir un renglón.** Aparece sola en el esquema, en la tabla y al agrupar por intervalos. No hay que acordarse de nada más.
- **Un nombre nuevo para algo que ya existe no da trabajo.** Si el archivo del mes que viene escribe la misma medición con otra errata, otro acento u otras unidades, el sistema la reconoce y la manda al mismo sitio, sin tocar una línea de código. Así fue como esas setenta variantes quedaron reducidas a una lista limpia.

En el código, el diccionario es `CONCEPT_MAP` y vive en el módulo `normalize`.

2.2 Módulos

Módulo	Responsabilidad única
normalize	slugify normaliza acentos, unidades y mayúsculas; CONCEPT_MAP traduce el slug al nombre canónico.
schemas	Deriva CANONICAL_COLUMNS, infiere etiquetas del nombre y decide el método de agregación por etiqueta.
extract	Lee el CSV tolerando filas irregulares, normaliza cabeceras, tipa y parsea el timestamp.
transform	split_streams separa el flujo eléctrico del de radiación <i>por columnas</i> ; remuestrea a 5 min y 15 s. Sin limpieza.
ddl	Genera tablas, vistas de corrección y calibración, diccionario y sentencias RLS.
clearsky	GHI de cielo despejado con pvlib (modelo Ineichen).
calibracion	Puebla radiacion_sc_clearsky de forma incremental.
performance	Calcula POA por arreglo (transposición Erbs + isotrópica, modelo bifacial de dos planos) y puebla radiacion_sc_poa.
load	UPSERT por timestamp en cada tabla.
state	Registro md5 en _ingest_log.
pipeline	Orquesta <i>extract</i> → <i>transform</i> → <i>load</i> y el refresco solar.
cli	Menú interactivo numerado.

2.3 Idempotencia

Dos niveles, deliberadamente redundantes:

- **Nivel archivo:** _ingest_log guarda el md5 de cada CSV. Si no cambió, ni se abre.
- **Nivel fila:** timestamp es PRIMARY KEY con ON CONFLICT DO UPDATE. Reprocesar nunca duplica, aunque el md5 falle.

Un archivo que revienta no frena la corrida: se registra el fallo, se hace *rollback* de esa transacción y el pipeline sigue con el siguiente. La capa solar va en su propio try, para que un problema de pvlib no invalide una carga de datos que ya funcionó.

2.4 Uso

Un solo comando, menú interactivo:

```
python3 main.py
```

Las opciones van en orden de confianza creciente: auditar el dataset, ejecutar en seco sin tocar la base, generar el DDL, aplicarlo en Supabase, y recién ahí cargar datos (incremental o reprocesando todo). Para archivos nuevos basta soltarlos en dataset/Monitoreo-AgroVoltaic-SC-NEW/ y repetir la carga incremental.

Lo que este ETL todavía no hace

La **separación fina de filas mezcladas** (el llamado Paso 2) sigue pendiente. Hoy se conservan las filas que coinciden con la cabecera del archivo y las irregulares se saltan con registro en el log. El criterio acordado es recuperar lo que se pueda y aceptar huecos; existe un par de archivos original/corregido como referencia.

3 Modelo de datos

Todo vive en un único proyecto Supabase (`jijklguopafevyucogro`). El modelo tiene cuatro capas: el crudo se guarda intacto y cada capa siguiente es una vista que agrega información sin destruir la anterior.

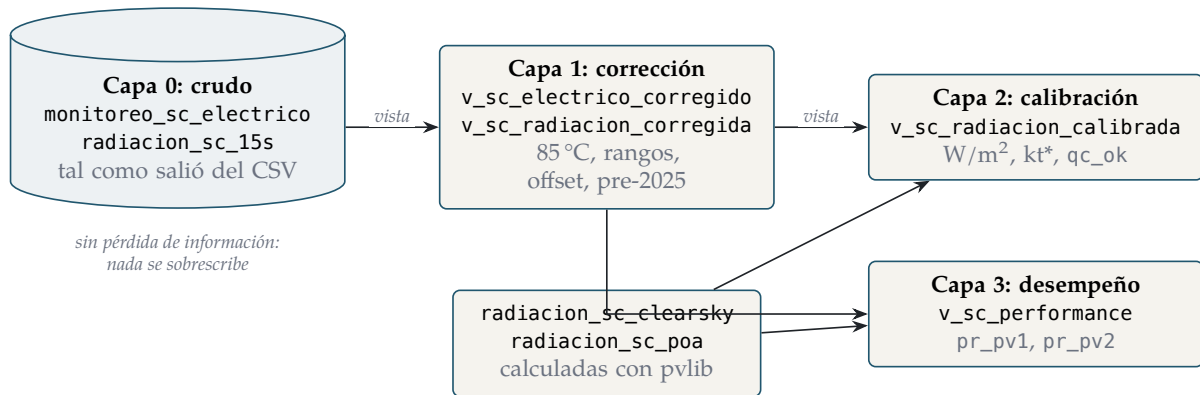


Figura 3: Capas del modelo. Una consulta puede pedir el dato tal como salió del sensor o el dato corregido y calibrado, según lo que necesite justificar.

3.1 Relaciones

Relación	Tipo	Contenido
monitoreo_sc_electrico	tabla	Crudo eléctrico a 5 min: voltaje, corriente, potencia y energía por arreglo, salida AC, temperatura del inversor y de cada arreglo.
radiacion_sc_15s	tabla	Crudo de radiación a 15 s: incidente, reflejada, albedo, y las mismas del piranómetro SP722.
radiacion_sc_clearsky	tabla	cs_ghi_wm2 por instante, calculado con pvlib.
radiacion_sc_poa	tabla	Irradiancia en el plano del arreglo, frontal y bifacial, para PV1 y PV2.
diccionario_variables	tabla	Definición en prosa de cada variable.
_ingest_log	tabla	md5 y conteo por archivo procesado.
v_sc_electrico_corregido	vista	Aplica rangos válidos; fuera de rango pasa a NULL.
v_sc_radiacion_corregida	vista	Neutraliza el <i>offset</i> , recorta negativos y anula el período inválido; marca <i>valido</i> .
v_sc_radiacion_calibrada	vista	Radiación en W/m², kt_star y bandera qc_ok.
v_sc_performance	vista	pr_pv1 y pr_pv2: Performance Ratio por arreglo.
lecturas_ambientales_sc	tabla	Store de la ingesta ambiental, formato largo.
predicciones	tabla	Auditoría de cada pronóstico emitido.
agente_log	tabla	Log estructurado del ETL y de los agentes.
gasto_diario	tabla	Gasto acumulado en el LLM, por día.

3.2 Reglas de corrección

Los umbrales son parámetros en `config.py` y el SQL de las vistas se genera a partir de ellos, así que cambiar un límite es cambiar una constante y regenerar el DDL.

Variable	Rango válido	Motivo
Temperaturas	10–80 °C	Criterio de Leo Cardinale; además el valor exacto 85,0 es el código de error del DS18B20 desconectado.
Potencia por string	0–5000 W	Cota física provisional; el instalado real es 1420 Wp por arreglo.
Voltaje de string	0–600 V	Rango del inversor.
Corriente de string	0–20 A	Rango del inversor.
Frecuencia de red	55–65 Hz	Red de 60 Hz.
Tensión AC	100–280 V	Salida monofásica.
Albedo	0–1	Definición del cociente.
Irradiancia	no aplica	<i>Offset</i> –38,845 → 0; negativos → 0; todo lo anterior al 2025-07-01 pasa a NULL (error de sensor corregido a mediados de 2025).

3.3 Calibración y desempeño

No hay constante de calibración de fábrica: lo que se creía una “celda calibrada” resultó ser un nombre comercial. Se calibra entonces contra el cielo despejado, comparando lo que mide el sensor con lo que debería medir un día sin nubes. Los datos del período válido ya venían en W/m^2 , así que la escala quedó en 1,0 y lo que aporta esta capa es el índice de claridad y el control de calidad.

Toda la capa se reduce a dos fórmulas. La primera dice cuánta de la luz posible llegó de verdad:

$$kt^* = \frac{GHI_{\text{medida}}}{GHI_{\text{cielo despejado}}} \quad \text{sólo donde } GHI_{\text{cs}} > 20 \text{ W}/\text{m}^2$$

La segunda compara la potencia real con la que cabría esperar de la luz que recibe el plano del arreglo:

$$PR = \frac{P/P_0}{POA/1000} \quad P_0 = 1420 \text{ Wp por arreglo}$$

Como los paneles son bifaciales, la POA suma las dos caras: frontal + φ ·trasera, con $\varphi = 0,80$ y el albedo medido.

El resultado da confianza porque converge: dos arreglos con geometrías muy distintas aterrizan casi en el mismo número, PV1 inclinado en 0,622 y PV2 vertical en 0,626.

Geometría confirmada del sitio

1420 Wp por arreglo ($4 \times 355 \text{ Wp}$), 2840 Wp en total, ambos bifaciales. PV1 es el arreglo inclinado (*tilt* 20°, *azimut* 150°); PV2 es el vertical (*tilt* 90°, *azimut* 50°), con Norte = 0° y sentido horario positivo. Zona horaria UTC–6 sin horario de verano.

4 Agente Predictivo

Pronostica irradiancia y humedad de suelo para San Carlos. Es un agente LLM con herramientas, pero el reparto de trabajo es estricto.

El LLM nunca toca los números

Claude hace cuatro cosas: entiende la pregunta, traduce el horizonte a segundos, llama a una herramienta que hace el cálculo físico, y redacta la respuesta con su incertidumbre. Los números salen de un modelo con anclaje físico, no de la intuición del modelo de lenguaje.

Así la parte numérica es auditable y reproducible, y el LLM aporta lo que sí sabe hacer: orquestación, ruteo y explicación.

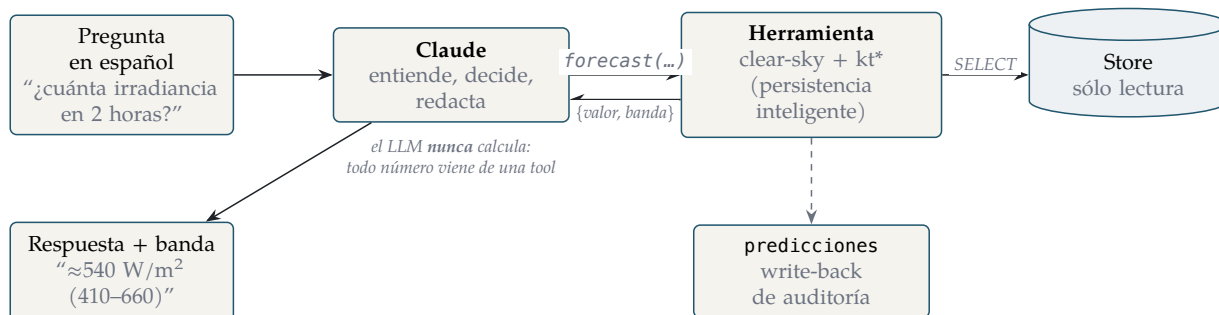


Figura 4: El lazo de *tool-use*. La flecha de vuelta lleva un diccionario con valor y banda; el LLM sólo lo narra.

4.1 La física

Pronosticar la irradiancia de frente es difícil, pero por un motivo que se puede quitar de en medio: el número lo domina la curva del sol a lo largo del día, que es pura astronomía y se conoce de antemano. Lo que de verdad cuesta predecir son las nubes. Así que se separan las dos cosas:

$$GHI_{medida} = kt^* \times GHI_{cielo \text{ despejado}}$$

De esos dos factores, el segundo se calcula y el primero es el único que hay que pronosticar.

Y no se pronostica repitiendo el kt^* reciente, que sería lo obvio y es lo que se hacía al principio. Se persiste su *anomalía* respecto de lo normal a esa hora del día, trasladada a lo normal de la hora que se pronostica:

$$kt_{pron}^* = kt_{clima}^*(t + h) + w \cdot (kt_{reciente}^* - kt_{clima}^*(t))$$

Son dos correcciones distintas por dos motivos distintos. La primera, porque en San Carlos las tardes se cierran por convección: trasladar la mañana a la tarde como si fueran iguales producía un sesgo medido de +52 % a las 16 h. La segunda, el peso w , porque creerle a lo reciente vale cada vez menos a medida que el horizonte crece: la correlación del índice consigo mismo cae de 0,90 a los 5 minutos a 0,13 a las 6 horas. El peso se mide sobre el histórico, no se declara.

Reconstruir es volver a multiplicar, y ahí entra el cielo despejado del instante futuro, que se calcula y no se adivina:

$$GHI_{pred}(t + h) = kt_{pron}^* \times GHI_{cs}(t + h)$$

Ahí está la diferencia con repetir la última lectura: si la hora pronosticada cae más cerca del mediodía, el pronóstico sube aunque las nubes sigan igual.

Para humedad de suelo no hay curva del sol que descontar. Se persiste la mediana reciente con una banda por variabilidad, y alcanza porque el suelo cambia lento.

El detalle de cada fórmula, incluido cómo se deriva el peso y de dónde sale la banda, está en el documento *Formulario*, que las reúne todas con el archivo donde vive cada una.

Barrera anti-fuga

`get_recent_data(now, lookback)` sólo devuelve lecturas con `timestamp < now`. Es una sola función, en una sola capa (`data.py`), y es la que hace honesto al *backtest*: el reloj se simula y el pronosticador jamás ve el futuro que se le está pidiendo predecir.

4.2 Estructura y dirección de dependencias

Las capas dependen sólo hacia abajo. Nadie importa hacia arriba:

```
config → domain → {data, physics} → forecasters → tools → agent → cli
```

El valor de esa disciplina se ve al sustituir piezas: cambiar el sitio o el modelo es tocar config; agregar un pronosticador serio (AutoARIMA, ML) es implementar la interfaz `forecast(variable, horizon_seconds, now)` sin tocar nada aguas arriba.

4.3 La ingesta ambiental

Un ETL propio, distinto del de CSV, trae datos de AgroDash al store:

- **Fuente y destino son distintos a propósito:** DATABASE_URL apunta a AgroDash (sólo lectura) y STORE_URL a Supabase. El store desacopla el pronosticador de la fuente, le da historia propia y aloja predicciones y logs.
- **Idempotente e incremental:** ON CONFLICT (origen_id) DO NOTHING, con marca de agua igual al máximo ts del store menos un solape.
- **Escalable en memoria:** cursor del lado del servidor y COPY a tabla temporal en lotes de 50 000. Los lotes chicos existen porque un COPY único de 577 000 filas chocaba contra el statement_timeout de Supabase.
- Los *timestamps naive* de AgroDash se etiquetan como hora local de Costa Rica, lo que se verificó viendo el pico diurno de irradiancia caer a las 11h.

4.4 Endpoints

Ruta	Para qué	Clave
GET /health	Ping para monitoreo.	no
GET /salud/ingesta	Antigüedad de los datos por variable; responde 503 si están viejos, para que un <i>healthcheck</i> lo note por código de estado.	no
GET /salud/panel	Lo anterior más errores recientes, gasto del día y última predicción.	sí
POST /forecast	Pronóstico a un horizonte; audita en predicciones.	sí
POST /anomalias	Detección determinista, sin LLM.	sí
POST /preguntar, /chat	Lazo completo del LLM, con traza.	sí
GET /serie, /backtest, /uso	Datos y consumo.	sí

Las dos rutas abiertas lo están por diseño: son de monitoreo automático y no exponen datos de la serie, sólo edades y estado.

4.5 Frenos de consumo

Sin tope, un bucle o un *scraper* dispara el costo del LLM. Hay tres frenos: límite de ritmo por identidad en los endpoints que gastan tokens (12 por minuto), límite más laxo en los deterministas (120 por minuto, que sólo protege CPU), y presupuesto diario en dólares. El gasto se acumula en la tabla `gasto_diario` del store, no en un archivo local: así sobrevive a que se recree el contenedor y no se duplica si hay más de una instancia corriendo.

4.6 Detección de anomalías

Primer ladrillo de la futura capa de agentes, y deliberadamente sin LLM. Dado (variable, ventana) devuelve hallazgos tipificados: `sin_datos_recientes`, `sensor_plano` (el patrón de los 85 °C), `fuera_de_rango`, `outlier` (puntuación robusta por mediana y MAD) y `drift`. La señal analizada es `kt*` para irradiancia (reutiliza el mismo `clear-sky`) y el crudo para humedad. El LLM lo llama como herramienta y sólo narra los hallazgos.

5 Agente Histórico

Responde preguntas en español sobre el histórico fotovoltaico ya limpio. Mismo principio que el forecaster: el LLM entiende y redacta, los números salen de consultas SQL de sólo lectura contra las vistas corregidas y calibradas.

5.1 Una herramienta, un archivo, una pregunta

La diferencia con el pronóstico está en la granularidad. Aquí cada herramienta responde *una* pregunta concreta y el LLM **compone** varias cuando la pregunta es compuesta. No hay una herramienta genérica de “ejecutá este SQL”, que sería la vía rápida y también la que abre la puerta a inyección y a respuestas improbables.

Herramienta	Qué responde
energia	Energía generada por arreglo (integral de potencia).
performance	Performance Ratio por arreglo, con el modelo bifacial.
irradiancia	GHI, <code>kt*</code> e insolación, con control de calidad.
temperatura	Temperatura por arreglo.
cobertura	Qué datos hay: rango temporal y conteos.
catalogo	Diccionario de variables.
tendencia	Evolución de una variable en el tiempo.
graficar	Serie lista para dibujar.
calidad_periodo	Veredicto de calidad: días utilizables, degradados y sin datos.
hallazgos_calidad	Que esta mal y donde, por tipo de problema y severidad.
cielo_periodo	Como estuvo el cielo: <code>kt</code> medio, variabilidad y reparto de días.

Las tres ultimas llegaron de fusionar lo que iba a ser un agente aparte, el comparador, cuando quedo claro que no era otro agente sino tres herramientas mas del mismo: responden sobre el mismo historico y contra la misma base.

El registro (`tools/__init__.py`) es puro ensamblado: junta los esquemas que ve el modelo y el mapa nombre → función. Agregar una herramienta es crear su archivo e importarlo ahí; no hay lógica que actualizar en dos lugares.

5.2 Endpoints

Ruta	Para qué	Clave
GET /health, /tools	Ping y esquemas de las herramientas.	no
POST /tool/<nombre>	Ejecutar una herramienta directo, sin LLM.	sí
POST /preguntar, /chat	Lazo completo del LLM, con traza.	sí
GET /datos/*	Vistazo de sólo lectura a las relaciones permitidas.	sí

Exponer cada herramienta como endpoint HTTP tiene un motivo concreto: permite verificar un número sin gastar tokens, y permite que un orquestador externo la cablee como nodo de acción en vez de depender del lazo conversacional.

El borde de seguridad de /datos/*

Esos endpoints interpolan nombres de tabla dentro del SQL, porque un identificador no se puede parametrizar. Por eso la **lista blanca** de `datos.py` es el borde de seguridad real: agregar una relación es agregarla ahí, nunca aceptar el nombre desde afuera. Hay pruebas que fijan esa regla para que no se relaje por descuido.

6 Consola de depuración

Una web mínima en Next.js para probar los dos agentes en vivo, con datos reales. No es un producto: es el instrumento con el que se verifica que un agente no está inventando.

6.1 La regla de verificación

Por cada pregunta, la consola dibuja la traza completa: cada turno del modelo con su texto y las herramientas que decide llamar, cada ejecución real con su entrada, su **salida cruda**, su tiempo en milisegundos y su error si lo hubo, y al final la respuesta redactada con el consumo de tokens y el costo.

Regla de oro

Todo número de la respuesta final tiene que aparecer en la salida de alguna herramienta. Si aparece un número que no está en ninguna salida, el modelo lo alucinó.

6.2 Camino de un request

Navegador → `/api/historico/*` → `:8010` → Supabase (RO)

Navegador → `/api/predictivo/*` → `:8000` → store (RO)

El navegador **nunca** habla directo con los servicios Python ni ve las API keys. Todo pasa por manejadores de ruta del lado servidor, que reenvían la petición inyectando la clave. Las claves viven sólo en la configuración del servidor.

Los endpoints que la consola necesita se agregaron a los propios agentes, no se reimplementaron en Node. Es una decisión de fuente única de verdad: si el lazo del LLM se reimplementara en el cliente, habría dos lazos que mantener sincronizados y la traza dejaría de reflejar lo que realmente corre en producción.

6.3 Control de acceso

El *middleware* decide si un request pasa; no sabe firmar cookies ni renderizar el login. Las páginas redirigen al login; las rutas `/api/*` responden 401 en JSON, porque un fetch no puede hacer nada útil con una redirección a HTML.

Fallar cerrado

Sin `DEBUGGER_PASSWORD` configurada, en producción la consola responde **503** en lugar de abrirse. Fallar abierto fue exactamente lo que la dejó expuesta en Amplify en su momento. En desarrollo sí deja pasar, para no estorbar el trabajo local. `DEBUGGER_SESSION_SECRET` firma la cookie: rotarlo cierra todas las sesiones sin cambiarle la contraseña al equipo.

6.4 Dos formas de levantarla

Desde el despliegue hay una tercera, y es la normal: entrar a la consola publicada. Las otras dos siguen existiendo porque sirven para otra cosa.

`./dev.sh` monta todo local: los dos agentes y la web, contra las bases reales en sólo lectura. Es lo que se usa para desarrollar.

`./consola.sh` abre un túnel SSH contra los agentes que *ya corren* en el servidor y levanta la web local contra ellos. Sirve para lo que la consola desplegada no puede: probar código propio todavía sin desplegar contra los datos de producción. Elige puertos libres (8000, 8010 y 3000 suelen estar ocupados en una máquina de trabajo), sincroniza las URLs con los puertos de esa corrida y cierra el túnel al salir.

7 Despliegue y operación

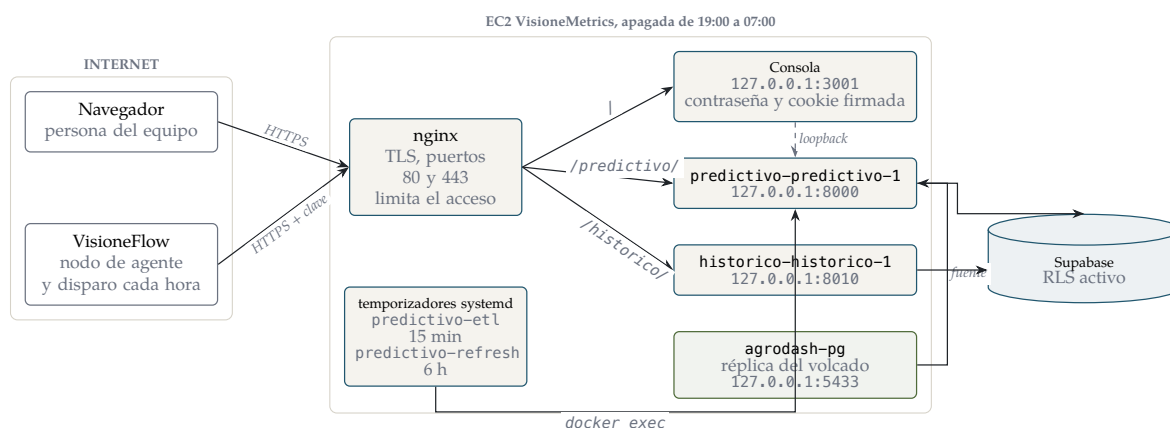


Figura 5: Qué corre dónde. Los dos agentes y la réplica escuchan sólo en el *loopback*; lo único publicado es nginx, con TLS, y detrás de él la consola con su control de acceso y las dos rutas de servicio que consume VisionFlow.

7.1 Por qué la plataforma vive en su propio servidor

Hasta el 2026-08-24 los dos microservicios corrían *dentro* de la EC2 de VisionFlow, la plataforma de agentes de la empresa. Funcionaba, pero ataba dos proyectos que no tienen nada que ver: el despliegue del pronóstico vivía en el repositorio de la plataforma, cada despliegue del *back-end* hacía `git reset --hard` y había que acordarse de no tocar ciertos archivos, y una caída de cualquiera de los dos arrastraba al otro.

Desde entonces la plataforma de datos tiene servidor propio, dominio propio y certificado propio. VisionFlow le sigue hablando, pero ahora como lo que es: una API externa, con su clave, sobre HTTPS. Es la misma relación que tendría con cualquier proveedor de terceros, y esa es exactamente la idea.

7.2 Qué corre dónde

Pieza	Dónde	Cómo sobrevive a un reinicio
Agente Predictivo	predictivo-predictivo-1 127.0.0.1:8000	restart: unless-stopped
Agente Histórico	historico-historico-1 127.0.0.1:8010	ídem
Réplica de AgroDash	agrodash-pg 127.0.0.1:5433	ídem, sobre volumen persistente
Consola	proceso Node 127.0.0.1:3001	gestor de procesos con estado guardado
Borde HTTPS	nginx del sistema puertos 80 y 443	unidad systemd habilitada
Ingesta cada 15 min	predictivo-etl.timer	unidad systemd habilitada
Refresco cada 6 h	predictivo-refresh.timer	ídem
Almacenamiento	Supabase	servicio gestionado

El servidor no está encendido las 24 horas

Un programador de tareas de AWS apaga la instancia a las 19:00 y la enciende a las 07:00, de lunes a viernes. De viernes a lunes está apagada el fin de semana completo. Es una decisión de costo, no un fallo, pero condiciona todo lo demás: cualquier consumidor externo tiene que disparar dentro de esa ventana. La ingesta no pierde nada porque sus temporizadores llevan `Persistent=true` y `OnBootSec`, así que al encender se pone al día sola con lo que se perdió.

7.3 Qué se publica, y qué no

Sólo hay tres cosas alcanzables desde internet, todas por el mismo nginx con TLS:

Ruta	Va a	Quién puede
/	la consola	Quien tenga la contraseña. Sin sesión válida, las páginas redirigen y <code>/api/*</code> responde 401.
/predictivo/...	el Predictivo	Quien tenga la clave de servicio. No pasa por el control de la consola: un flujo automático no tiene sesión.
/historico/...	el Histórico	ídem.

Los nombres anteriores (`/forecast/` y `/analizador/`) siguen respondiendo como alias, para que el cambio de servidor no obligara a reconfigurar rutas en el canvas de la plataforma.

7.4 Por qué hay una réplica de AgroDash

El servidor de Cartago está caído, así que la fuente de la ingesta pasó a ser una réplica restaurada de un volcado, montada en el propio servidor. El puerto queda atado al loopback y ocupa unos 6GB. Volver a la fuente viva es cambiar una línea de configuración: la URL de Cartago quedó comentada, no borrada.

Un detalle que cuesta tiempo si no se sabe: si el `pg_restore` de la máquina es más nuevo que el servidor, falla con `unrecognized configuration parameter "transaction_timeout"`. Por eso la restauración se hace con el `pg_restore` de *adentro* del contenedor.

Al mudar el servidor, la réplica se movió por la red privada de la nube, sin salir a internet, y se verificó contando las filas de las dos puntas: 21 314 662 lecturas en el origen y las mismas 21 314 662 en el destino, con idéntica marca de tiempo máxima. Un `pg_restore` que termina con código cero no es prueba de nada; el conteo sí.

7.5 Salud del sistema

Se mira sin entrar al servidor, por HTTP: `/salud/ingesta` da la antigüedad de los datos por variable (503 si están viejos) y `/salud/panel` agrega errores recientes, gasto del día y última predicción. La consola los muestra en su sección de salud.

El 503 actual es correcto

La fuente de San Carlos está congelada desde el 2026-07-23: el subsistema de cajas con sufijo SC y los nodos de suelo dejaron de reportar. `/salud/ingesta` responde 503 porque está informando la antigüedad *real* de los datos, no porque el agente falle. Todo corre en modo “sólo histórico”; cuando el equipo restaure la ingesta, el ETL rellena solo por ser idempotente e incremental.

7.6 Lección de observabilidad

El ETL falló cada 15 minutos durante nueve días **sin dejar rastro** en `agente_log`. La causa: la conexión a la fuente estaba *fuera* del bloque `try`, así que la excepción se propagaba antes de que hubiera con qué registrarla. La corrección tiene tres partes, y las tres importan: conectar primero al store (que es donde se escribe el log), meter la conexión a la fuente dentro del `try` con su propio evento de fallo, y **volver a lanzar** la excepción para que `systemd` siga marcando la unidad como fallida. Hay una prueba de regresión que fija ese comportamiento.

7.7 Diagnóstico rápido

Síntoma	Dónde mirar
La ingesta no trae datos	<code>systemctl status predictivo-etl.service</code> y la tabla <code>agente_log</code> filtrando por nivel de error.
El pronóstico responde datos viejos	<code>GET /salud/ingesta</code> : la fuente está congelada, no es un fallo del agente.
Nada responde, de noche o en fin de semana	El servidor está apagado por programación. No es una caída.
Todo responde 429	Tope diario de gasto, límite de ritmo del servicio, o el freno de <code>nginx</code> si es el acceso a la consola.
La consola responde 503	Falta <code>DEBUGGER_PASSWORD</code> en el entorno. Falla cerrada a propósito.
Un servicio da 502	El contenedor está caído; <code>nginx</code> sigue en pie.
Supabase rechaza escrituras	El plan gratuito son 500 MB y está cerca del límite.

7.8 Pruebas

Ninguna suite necesita credenciales ni red hacia Supabase; si una empieza a pedir las, dejó de ser unitaria. Corren en CI en cada *push* y cada PR.

Componente	Pruebas	Qué cubren
Agente Predictivo	253	Horizonte, física, herramienta, ETL, límites, salud, claves, riesgo, climatología, observabilidad, API.
Agente Histórico	61	API, costos, lista blanca de datos, control de calidad, cielo, contexto.
Consola	3 guiones de humo	Construcción, control de acceso y formato de respuestas.
ETL de CSV	ninguna	Sin suite; la verificación es el <i>notebook</i> de EDA.

8 Seguridad

- **AgroDash es sólo lectura, y se fuerza.** Toda conexión fija `SET SESSION CHARACTERISTICS AS TRANSACTION READ ONLY` y sólo se ejecutan `SELECT`. No se confía en que el rol esté bien configurado del otro lado.
- **RLS en modo cierre** en las tablas públicas de Supabase, deliberadamente *sin políticas*: sólo los roles de servicio acceden y la API REST pública queda bloqueada. Se verificó que no rompe al forecaster, que corre con un rol que la salta.
- **Vistas con `security_invoker`**, para que ejecuten con los permisos de quien consulta y no con los del dueño.
- **Claves sólo por entorno**, nunca en el código ni en el repositorio. Ningún secreto está versionado.
- **Comparación en tiempo constante** de las API keys (`secrets.compare_digest`), para no filtrar la clave por diferencias de tiempo de respuesta.
- **Clave exigida sólo si está configurada.** En local los servicios corren abiertos para no estorbar; en producción la variable está definida y el borde exige el encabezado. La excepción son `/health` y `/salud/ingesta`, abiertas a propósito para monitoreo automático: no exponen datos de la serie.
- **Los servicios no se publican; se publica el borde.** Los dos agentes y la réplica escuchan en `127.0.0.1` y nada más. Lo único alcanzable desde internet es nginx, con TLS. Hasta la mudanza los servicios escuchaban en `0.0.0.0` y lo único que los tapaba era el grupo de seguridad de la nube: cualquier máquina de la misma red privada los alcanzaba. Atarlos al loopback quita esa clase entera de error.
- **Freno de fuerza bruta en el borde.** El límite de intentos de la propia consola vive en memoria del proceso, así que un reinicio lo borra y en un despliegue con varias instancias no se comparte. La defensa que no depende de eso es doble: cada intento cuesta un derivado de clave deliberadamente caro (unos 400 ms de CPU), y nginx limita la ruta de acceso a seis intentos por minuto. Verificado, no sólo configurado: diez intentos seguidos dan seis 401 y cuatro 429.
- **Falla cerrada.** Sin contraseña configurada, la consola en producción no se abre: responde 503 y lo dice. Un despliegue anterior quedó expuesto justamente por hacer lo contrario, abrirse cuando faltaba la variable.
- **Errores internos genéricos.** Un fallo interno responde el 500 estándar; el detalle queda en el log del servidor, nunca en la respuesta.

8.1 Secretos necesarios

Variable	Para qué
<code>DATABASE_URL</code>	Conexión a Supabase (usar el <i>Session pooler</i> ; la conexión directa es sólo IPv6).
<code>STORE_URL</code>	El store del agente, separado de la fuente a propósito.
<code>ANTHROPIC_API_KEY</code>	Los dos agentes.
<code>FORECAST_API_KEY</code>	Proteger el servicio de pronóstico.
<code>HISTORICO_API_KEY</code>	Proteger el servicio del Histórico.
<code>DEBUGGER_PASSWORD</code>	Entrar a la consola.
<code>DEBUGGER_SESSION_SECRET</code>	Firmar la cookie de sesión.

8.2 Pendiente de seguridad

Rotar la clave débil del rol de sólo lectura que se usó para probar el acceso a la base viva de Cartago por la malla Tailscale.

9 Estado actual

9.1 Volúmenes actuales

Relación	Filas	Nota
monitoreo_sc_electrico	36 469	285 CSV, ventanas de 5 min.
radiacion_sc_15s	94 868	Ventanas de 15 s.
radiacion_sc_clearsky	94 868	Un cielo despejado por instante.
radiacion_sc_poa	56 450	Instantes con POA por arreglo.
lecturas_ambientales_sc	≈ 812 000	Ingesta de AgroDash, formato largo.

La verificación cruzada confirmó que el crudo sigue siendo crudo (temperaturas en 85 y el *offset* presentes en la tabla base) y que las vistas hacen su trabajo (85 pasa a NULL, la potencia máxima cae a un valor físico, la irradiancia negativa queda en cero, kt* con percentil 95 en 1,01 y 99,4 % de filas aprobando el control de calidad).

9.2 Decisiones que conviene no revisitar

- El crudo se guarda; la corrección vive en vistas. Cualquier propuesta de “limpiar al insertar” ya se descartó explícitamente.
- No se generan datos sintéticos para los huecos largos de 126 y 71 días. Como referencia paralela se usa NASA POWER.
- Los duplicados exactos por md5 se eliminan; los fragmentos numerados requieren decisión humana.
- Las bases de las dos regiones no se fusionan a nivel de almacenamiento, aunque un agente lea ambas.

Dónde seguir

El sistema de memoria del proyecto (docs/memoria/INDEX.md) es la fuente viva: un tema por archivo, con la evidencia contada y la fecha de cada decisión. Este documento resume la arquitectura; ese índice explica *por qué* cada pieza quedó así.